

Integración de Sentry

ProjectFlow · React, Vite, Axios y TanStack Query

Documento técnico y guía de mantenimiento Estado del código revisado: 2 de octubre de 2026

Este documento explica qué se ha integrado, cómo funciona, por qué se eligió cada mecanismo y cómo comprobarlo. Describe el código del proyecto, no una configuración genérica de Sentry.

La integración actual captura errores de React, rutas y operaciones gestionadas con TanStack Query. Mide navegación y peticiones desde el navegador y adjunta una copia depurada del payload cuando una petición Axios falla.

La configuración final incluye errores HTTP 4xx, como el 401 que impedía crear productos. Las cancelaciones voluntarias se excluyen. Los errores de autenticación se pueden registrar, pero sus payloads completos se omiten.

No se han incluido valores reales del archivo .env, tokens, contraseñas ni información de usuarios. Los ejemplos de eventos son ilustrativos, no exportaciones del panel.

Lectura recomendada: primero las secciones 1 a 4 para entender el sistema; después las secciones 5 a 10 para revisar implementación y privacidad; finalmente las secciones 11 a 14 para validar y mantenerlo.

Índice y alcance del documento

1. Qué se ha integrado y qué significa «todas las peticiones» 2. Arquitectura y recorrido completo de un fallo 3. Inicialización y configuración de Sentry 4. Captura centralizada de consultas y mutaciones 5. Clasificación, contexto y prevención de duplicados 6. Errores de React, rutas y recuperación visual 7. Rendimiento, navegación y peticiones HTTP 8. Usuario autenticado y ciclo de sesión 9. Payloads: extracción, exclusiones y límites 10. Privacidad y límites de las garantías actuales 11. Ejemplo completo: crear un producto devuelve 401 12. Pruebas realizadas y verificaciones pendientes 13. Diagnóstico y procedimiento de comprobación 14. Inventario, mantenimiento y decisiones de diseño

Cada sección comienza en una página nueva. El documento distingue tres conceptos: lo implementado en código, lo probado localmente y lo que necesita validación en un navegador o en el proyecto remoto de Sentry.

Fuentes utilizadas: módulos de `src/infrastructure/monitoring`, entrada React, configuración del router, hooks de presentación, repositorios Axios y `tests/sentry.test.mjs`. Se conserva «infrastructure» porque es el nombre real de la carpeta del proyecto.

La documentación corresponde a esta instantánea. Si se cambian filtros, hooks, rutas o políticas de payload, se debe actualizar junto con el código.

1. Qué se ha integrado

Objetivo

Permitir investigar qué operación falló, con qué estado HTTP, en qué ruta y para qué ID de usuario, sin adjuntar indiscriminadamente la configuración de Axios. Cuando existe un cuerpo JSON de petición, se añade una copia limitada y depurada para ayudar a reproducir el problema.

Tres clases de información

Una incidencia procede de una excepción enviada a Sentry. Varios eventos pueden agruparse en una misma incidencia; no debe esperarse una tarjeta nueva por cada clic fallido.

Una traza describe trabajo y tiempos: carga de página, navegación y operaciones instrumentadas del navegador. Una petición correcta puede participar en una traza sin ser un error.

Un breadcrumb es contexto anterior a un evento, por ejemplo una navegación o metadatos HTTP. No equivale a una incidencia independiente ni a un registro exhaustivo y permanente de todas las acciones.

Cobertura actual

Los hooks existentes cubren productos, tareas, categorías, login y registro. Los fallos de sus consultas y mutaciones pasan por una caché global. Las fronteras de React y del router cubren errores de renderizado y rutas. La entrada React tiene manejadores para errores no capturados y recuperables.

También permanecen las integraciones por defecto del SDK; no se han sustituido por una lista vacía. La aplicación añade instrumentación de navegación y filtros antes del envío.

Qué NO significa «todas»

No hay un interceptor de respuesta Axios que reporte cada petición. Una llamada nueva ejecutada fuera de TanStack Query y cuyo error se capture y silencie localmente no llegará por esta vía. Tampoco se convierte automáticamente cada acción de interfaz, validación o cambio de estado en una incidencia.

No se instrumentó el servidor. Si una API responde 200 con un objeto que representa un fallo de negocio, la aplicación tendría que convertirlo explícitamente en error para que se detecte como tal. Una incidencia tampoco corrige el token ni vuelve válida una operación rechazada.

2. Arquitectura y recorrido de un fallo

Flujo principal de datos

Interfaz -> hook -> caso de uso -> repositorio -> instancia Axios -> API.

Si la promesa falla: QueryCache o MutationCache -> reportError -> contexto depurado -> Sentry.captureException -> filterEvent -> transporte del SDK -> proyecto Sentry.

La interfaz conserva su propio estado de error y sus mensajes. La observabilidad es un efecto adicional del fallo; no reemplaza las responsabilidades de presentación, dominio o comunicación HTTP.

Por qué se centralizó

Añadir captureException en cada componente, caso de uso y repositorio produciría repetición y facilitaría enviar varias veces la misma excepción. Centralizar en las cachés cubre los hooks existentes con un mismo criterio, sin acoplar el dominio al SDK.

Los casos de uso, como categoryUseCases.ts, siguen encargándose de las operaciones de la aplicación. No necesitan importar Sentry para cada llamada: el hook que los ejecuta ya participa en la infraestructura común.

Responsabilidades de los módulos

sentry.ts inicializa el cliente, configura trazas y conecta la sesión. queryClient.ts instala callbacks globales de errores. errors.ts decide cómo convertir el fallo en un evento y añade etiquetas y contexto.

payload.ts extrae y depura el cuerpo de una petición Axios. privacy.ts filtra eventos, breadcrumbs y spans justo antes de enviarlos. ErrorRecovery.tsx decide qué mostrar cuando falla el renderizado y comunica el error de esa frontera.

Aislamiento por evento

reportError utiliza withScope. Las etiquetas de recurso, operación y origen, así como el contexto de la petición, se aplican a esa captura. No se dejan como estado global para que otro evento herede por accidente el payload de una operación anterior.

El usuario sí pertenece al contexto de sesión y se sincroniza por separado. La misma infraestructura permite investigar varias operaciones del usuario sin incluir su token de acceso.

3. Inicialización y configuración

Orden de arranque

main.tsx importa primero el módulo de monitorización. Este inicializa Sentry antes de que App importe y cree el router instrumentado. El orden importa porque el seguimiento debe estar preparado al construir el router y arrancar la interfaz.

Se usa @sentry/react, declarado en package.json con el rango ^11.2.0. La integración se ajustó a los tipos instalados: se usa dataCollection y el formato de spans con name y attributes. La opción anterior sendDefaultPii no forma parte de esta configuración.

Variables y activación

El DSN se obtiene de import.meta.env.VITE_SENTRY_DSN. Identifica el destino de los eventos. enabled depende de Boolean(dsn): si falta o es una cadena vacía, se desactiva el envío. Esto no valida que un DSN no vacío sea correcto.

El entorno procede de import.meta.env.MODE. Permite separar los eventos según el modo de Vite. No se ha configurado una release propia ni un vínculo automático con commits o despliegues.

Parámetros principales

tracesSampleRate vale 1 en desarrollo y 0.1 fuera de desarrollo. tracePropagationTargets es una lista vacía. normalizeDepth es 10 para que el SDK preserve estructuras anidadas del payload que ya han sido limitadas por nuestro filtro.

La profundidad 10 no significa que se permita extraer un payload sin límites: sanitizePayload limita su propia recursión y volumen antes de entregarlo a Sentry.

Recolección automática

dataCollection desactiva userInfo, cookies, httpHeaders, httpBodies, urlQueryParams y stackFrameVariables. Los cuerpos no se recopilan automáticamente: existe una excepción explícita y controlada que copia el payload depurado al contexto request_payload.

No se habilitó Session Replay, grabación de pantalla ni un registro de todos los formularios. Tampoco se configuró carga automática de source maps. No hay que colocar un token privado de Sentry en variables VITE_: este código solo necesita el DSN del cliente.

4. Consultas y mutaciones

QueryClient compartido

QueryProvider crea una instancia de QueryClient mediante createQueryClient y la comparte con la aplicación. La QueryCache instala onError para consultas; MutationCache lo instala para mutaciones.

El callback de una consulta se ejecuta cuando la consulta termina en error después de sus reintentos. El módulo no impone un número propio de reintentos: respeta la configuración de TanStack Query o la de cada operación. La prueba local establece explícitamente dos reintentos para verificar este comportamiento.

La captura en la caché evita que cada componente que observa la misma consulta tenga que reportar el fallo por separado. Las mutaciones conservan sus callbacks locales, por ejemplo los que presentan un mensaje de error en un formulario.

Etiquetas instaladas

Productos: resource=products; operation=list, detail, create, update o delete.

Tareas: resource=tasks; operation=list, create o update. La existencia de otros métodos en un repositorio no implica que haya un hook instrumentado para ellos.

Categorías: resource=categories; operation=list o create.

Autenticación: resource=auth; operation=login o register.

La caché añade source=query o source=mutation. Se usan valores estables y pequeños; no se utilizan variables del formulario ni claves completas de consulta como etiquetas.

Por qué no se envían las variables de mutate

El payload se obtiene de error.config.data en Axios. Eso permite describir el cuerpo que la capa HTTP preparó y evita copiar ciegamente las variables de una mutación, que pueden incluir IDs, objetos auxiliares o credenciales no destinadas a enviarse.

Si una mutación falla antes de construir una petición Axios, habrá un error, pero no un payload HTTP extraído automáticamente. Las consultas GET normalmente no llevan cuerpo; en ese caso no se crea request_payload.

Para añadir una operación nueva, debe usar el QueryClient compartido y asignar meta.resource y meta.operation. Si se crea otro QueryClient sin estos callbacks, la operación no tendrá esta captura centralizada.

5. Tratamiento de errores y duplicados

Política final

Se reportan los errores HTTP 4xx y 5xx, los fallos de red sin respuesta y otros errores de JavaScript que lleguen a `reportError`. Se excluyen las cancelaciones reconocidas por Axios mediante `isCancel`.

Al principio se descartaban los 4xx. Esa decisión explicaba por qué el 401 de crear productos no aparecía como incidencia. La política se cambió por petición expresa: ahora también se capturan 400, 401, 403, 404, 422 y 429, entre otros.

Registrar un 401 de login no implica registrar sus credenciales: la política de captura de errores y la política de captura de payloads son independientes.

Conversión segura de Axios

No se entrega el `AxiosError` completo a `captureException` desde `reportError`. Se crea un `Error` con mensaje HTTP seguido del código, o `Network request failed` cuando no hay estado disponible. Se intenta conservar la pila sustituyendo su primera línea.

Esto evita adjuntar por accidente `config`, cabeceras o respuesta. El contexto `http` contiene `method`, `path` y `status`. El `payload`, si es admisible, se añade por separado y depurado.

Los errores normales conservan su objeto `Error`. Valores lanzados que no sean `Error` se convierten en un mensaje genérico: `Unexpected non-Error failure`. No se serializa arbitrariamente lo que se haya lanzado.

Dedución de la ruta

`cleanUrl` devuelve solo `pathname`, elimina consulta y fragmento y sustituye segmentos enteramente numéricos por `:id`. Por ejemplo, una URL absoluta terminada en `/products/123?token=...` se representa como `/products/:id`.

Alcance real de la deduplicación

Un `WeakSet` recuerda los objetos ya reportados. Si el mismo objeto vuelve a pasar por `reportError`, no se captura otra vez. Ayuda con efectos repetidos y con un error compartido.

No compara mensajes ni códigos HTTP. Dos peticiones distintas que produzcan objetos diferentes pueden generar dos eventos, aunque ambos digan HTTP 401. Tampoco garantiza deduplicación universal de rutas que envían directamente al SDK. El agrupamiento remoto de eventos es una función distinta y no se ha personalizado con fingerprints.

6. React, rutas y recuperación visual

Frontera de la raíz

`RootErrorBoundary` envuelve `QueryProvider` y `App`. Cuando un descendiente falla durante el renderizado, cambia a una pantalla de recuperación y comunica el error con `source=react-root`.

La pantalla informa de que no se pudo mostrar la página y ofrece Reintentar, que recarga el navegador, y Volver al inicio. El mensaje técnico y los datos de monitorización no se muestran al usuario final.

Frontera del router

Las rutas de login y registro y la rama protegida tienen `errorElement` con `RouteErrorRecovery`. Los errores de esa rama pueden ser consumidos por React Router antes de alcanzar la frontera exterior; por eso se necesita una captura dentro del router.

`useRouteError` obtiene el fallo. Si es una respuesta de error de ruta, se convierte en `Error` con mensaje `Route HTTP` y su estado. Se conserva una instancia estable mediante `useMemo` y se reporta desde `useEffect` con `source=router`. También se incluyen los 4xx de ruta.

Manejadores de la raíz React

`createRoot` incluye `onUncaughtError`, etiquetado `react-uncaught`, y `onRecoverableError`, etiquetado `react-recoverable`. No se añade una captura paralela en `onCaughtError`: las fronteras ya son responsables de sus errores manejados.

Qué no cubre una frontera por sí sola

Una frontera de renderizado no es un sustituto de la gestión de errores de peticiones asíncronas ni de todos los handlers de eventos. Los errores de API se cubren con `QueryCache` y `MutationCache`. Los fallos globales no manejados dependen de los mecanismos del SDK y del navegador.

`ErrorInfo` se recibe en `componentDidCatch`, pero no se adjunta expresamente su `componentStack` al contexto. El comportamiento visual y la ausencia de duplicados bajo todas las secuencias de navegación no se han comprobado mediante una prueba automatizada de navegador en esta sesión.

7. Rendimiento y navegación

Integración del router

Se usa `reactRouterBrowserTracingIntegration` con `useEffect`, `useLocation`, `useNavigationType`, `createRoutesFromChildren` y `matchRoutes`. La creación del router se envuelve con `wrapCreateBrowserRouter`.

La combinación permite instrumentar las navegaciones y asociar trabajo del navegador a trazas. No se ha escrito un span manual alrededor de cada caso de uso ni de cada botón.

Muestreo

En desarrollo se solicita una tasa de trazas de 1; en producción, 0.1. Esto reduce el volumen de rendimiento en producción y mantiene mayor visibilidad durante la depuración local.

El 10 % corresponde al muestreo de trazas, no a una promesa de que exactamente una de cada diez peticiones aparezca. Varias peticiones pueden pertenecer a una misma traza. Tampoco es una tasa de muestreo de errores: no se ha definido `sampleRate` para reducir la captura de excepciones.

La entrega real sigue dependiendo del estado del SDK, red, límites del proyecto y posibles bloqueos del navegador. Un porcentaje de configuración no garantiza recepción remota de cada evento elegible.

Sin propagación hacia el backend

`tracePropagationTargets=[]` evita que esta configuración añada cabeceras de propagación de trazas a las peticiones. Se decidió así porque no se había confirmado que el backend aceptara esas cabeceras por CORS.

Se puede medir desde el navegador sin crear una traza distribuida completa del servidor. No hay spans de base de datos, middleware o ejecución interna del backend aportados por esta intervención.

Depuración de spans

`filterSpan` limpia los atributos `url.full`, `http.url` y `http.target`. También depura partes del nombre cuando contienen una URL HTTP o una consulta. En la versión instalada se trabaja con `name` y `attributes`, no con `description` y `data`.

Las peticiones envelope observadas con estado 200 indican que un envío al servicio respondió correctamente; por sí solas no demuestran que contuviera una excepción. El sobre puede transportar otros tipos de información, como rendimiento.

8. Usuario y sesión

Identidad mínima

Sentry.setUser recibe únicamente un objeto con id convertido a cadena. No se añade el nombre, email, rol ni token desde la sincronización de sesión.

El identificador permite relacionar incidencias con una cuenta interna. Sigue siendo un dato que identifica a una cuenta dentro del sistema: «solo ID» no equivale a anonimato absoluto.

Sincronización inicial

El módulo consulta useAuthStore.getState().user al inicializarse. Así aplica el usuario disponible al arrancar, incluida la sesión que el store haya restaurado en ese momento.

Después se suscribe a cambios del store. Al iniciar sesión o cambiar el usuario, vuelve a establecer el ID. Cuando user pasa a null, ejecuta Sentry.setUser(null), eliminando esa asociación para futuros eventos.

La suscripción vuelve a leer el estado en cada actualización; no se transmite el store completo. Los eventos ya enviados no se modifican ni se borran al cerrar sesión.

Desarrollo con recarga de módulos

Si import.meta.hot está disponible, se registra la limpieza de la suscripción con dispose. Evita acumular suscripciones a Zustand cuando el módulo se reemplaza durante desarrollo.

Segunda defensa antes del envío

filterEvent reconstruye event.user conservando solo id si existe. Esto limita datos adicionales que pudieran llegar a ese campo por otras vías de eventos de error.

Alcance y verificación

Se utiliza el store de src/store/useAuthStore.ts, el mismo que usa la instancia Axios para obtener el token. No se lee el token para construir el usuario de Sentry.

La prueba de filtros verifica que un evento con id y email conserve únicamente el ID. La restauración visual de sesión, el inicio de sesión real y el cierre de sesión en el navegador necesitan comprobación funcional; no deben confundirse con esa prueba unitaria del filtro.

9. Payloads: qué se adjunta exactamente

Extracción

`getRequestPayload` solo actúa sobre errores reconocidos como Axios. Lee `config.data`. Si no existe, es `null` o es una cadena vacía, no añade contexto. Un objeto puede procesarse directamente; una cadena se intenta interpretar como JSON.

Las cadenas que superan 20.000 caracteres se sustituyen por `[Payload too large]`. Si no son JSON, se sustituyen por `[Non-JSON payload omitted]`. No se suben archivos ni se transforma automáticamente `FormData` en sus campos.

El resultado se guarda en `contexts.request_payload.body`. Es una copia para el evento: no modifica el cuerpo original ni la petición que usa la aplicación.

Exclusión de autenticación

Se omite todo el payload si `meta.resource` es `auth`. También si la URL contiene un segmento `auth`, `login`, `register`, `signin`, `signup` o `refresh` según el patrón implementado. Esto cubre las rutas actuales `/user/login` y `/user/register`, incluso si la captura no dispone de `meta`.

Una futura ruta de autenticación con otro nombre debe etiquetarse como `auth` o añadirse a la política. No existe reconocimiento semántico automático de todos los endpoints posibles.

Campos sensibles

Las claves se normalizan retirando caracteres no alfanuméricos y se comparan sin distinguir mayúsculas. Se filtran coincidencias con `password`, `passwd`, `pwd`, `token`, `authorization`, `cookie`, `secret`, `apikey`, `credential` y `email`.

Por ello `access_token` y `api_key` quedan filtrados. El valor se reemplaza por `[Filtered]`. También se filtran cadenas que parecen contener un Bearer token o un JWT conforme al patrón actual.

Límites de tamaño y estructura

Cada cadena conservada se corta a 500 caracteres. Cada array y objeto se limita a 50 elementos o propiedades. La recursión se corta por encima de profundidad 6 y hay un presupuesto compartido de 200 visitas; al agotarse se usa `[Truncated]`.

Referencias repetidas a objetos se marcan `[Circular]`, aunque no siempre formen un ciclo real. Solo se admiten objetos planos o sin prototipo, además de arrays y valores básicos. Otros objetos se marcan `[Unsupported payload]`; tipos no admitidos se marcan `[Omitted]`.

Estos límites contienen el volumen; no constituyen un límite exacto en bytes del evento completo, ni evitan por sí solos cualquier contenido personal escrito en texto libre.

10. Privacidad: defensas y límites

Antes de recolectar

`dataCollection` reduce la recolección automática de información de usuario, cookies, cabeceras, cuerpos HTTP, consultas de URL y variables de pila. La integración de breadcrumbs desactiva DOM y el filtro descarta categorías console y ui.*.

Antes de enviar

`filterBreadcrumb` conserva únicamente algunos campos HTTP y de navegación; limpia sus URLs y elimina `message`. `filterEvent` elimina `extra` y reconstruye `request` con solo `url` depurada y `method`. Reconstruye también `user` con solo ID.

Si el evento original es Axios, `filterEvent` sanea el mensaje de excepción y puede añadir el payload depurado. Es una defensa para capturas globales que no pasaron por `reportError`. Los eventos que sí pasaron por `reportError` ya llevan una excepción nueva y un contexto preparado.

`filterSpan` limpia determinados atributos URL de los spans de rendimiento. No se configura una captura general de cuerpos en las trazas: los payloads se añaden al evento de error.

Lo que estas defensas no garantizan

El filtro de claves no detecta toda información personal. Un campo `description` puede contener un teléfono, dirección, nombre o secreto escrito libremente y quedar conservado. El patrón de cadenas reconoce ciertos formatos de tokens, no todos los secretos posibles.

`cleanUrl` elimina `query`, fragmento y segmentos numéricos, pero no convierte automáticamente todos los UUID, nombres o emails en rutas anónimas. `filterEvent` no reescribe todos los mensajes de Error normales ni depura recursivamente todos los contextos ajenos al módulo.

El sanitizador limita la estructura, pero las claves mismas y campos de texto libres no constituyen una lista explícita de datos permitidos. Debe revisarse antes de usarlo con datos reales especialmente delicados.

Criterio de evolución

Si se necesita mayor control, una mejora sería una lista de campos permitidos por recurso, por ejemplo nombre y precio para productos, excluyendo texto libre. Esa mejora no está implementada y no debe confundirse con el filtro actual.

La activación de todos los 4xx responde a la necesidad de depuración del proyecto. En una aplicación con gran volumen puede requerir un criterio más selectivo; no se cambió aquí esa decisión del usuario.

11. Ejemplo: crear producto devuelve 401

Situación

Una persona pulsa guardar un producto. La petición POST /products recibe 401. El fallo puede deberse a un token ausente, caducado o rechazado; el estado por sí solo no permite distinguir la causa.

Secuencia

El repositorio rechaza su promesa a través de Axios. El fallo llega a la mutación de productos y MutationCache llama a reportError con resource=products, operation=create y source=mutation.

reportError no descarta 401. Obtiene método, ruta y estado; extrae el cuerpo de config.data, lo depura y construye un Error con mensaje HTTP 401. withScope adjunta el contexto solo a esa captura.

La interfaz sigue mostrando su mensaje local de error. Sentry no vuelve a ejecutar la operación ni renueva el token. El evento se filtra y se entrega al transporte configurado por el SDK.

Representación ilustrativa

Mensaje: HTTP 401 Etiquetas: resource=products; operation=create; source=mutation Usuario: id="42", si hay usuario en el store Contexto http: method="POST"; path="/products"; status=401 Contexto request_payload.body: name="Mesa"; price=25 Campo sensible de ejemplo: access_token="[Filtered]"

Si no había cuerpo, no existe request_payload. Si el endpoint era de autenticación, tampoco. Si el cuerpo era texto no JSON, aparece el marcador correspondiente en vez del texto original.

Dónde buscarlo

En el proyecto de Sentry, localizar la incidencia y abrir un evento reciente, no solo el resumen agregado. Revisar etiquetas, usuario y los contextos http y request_payload. El nombre exacto de las secciones puede variar en la interfaz remota.

Los errores anteriores al cambio de configuración no se reenvían retroactivamente. Para comprobar la integración nueva hay que recargar la aplicación y provocar un nuevo fallo. Un 401 repetido puede añadirse a una incidencia existente en lugar de crear otra.

12. Pruebas y evidencia

Prueba automatizada local

Comando: `node tests/sentry.test.mjs`

El test carga los módulos TypeScript con jiti y configura un transporte de Sentry en memoria. Utiliza un DSN ficticio y recoge los eventos; no valida la cuenta real ni necesita enviar las pruebas al proyecto remoto.

La ejecución se repitió al preparar este documento y terminó correctamente. El nombre resumido del mensaje de salida no enumera todas las aserciones; las pruebas incluyen también payloads.

Casos comprobados

Cancelaciones sin evento. Errores 400, 401, 403, 404, 422 y 429 enviados. Reenvío del mismo objeto sin duplicación. Captura de error 500 y fallo de red. Etiquetas de operación y recurso.

Depuración de URLs, eliminación de breadcrumbs de consola, eliminación de email y extras en eventos. Depuración de URLs en spans. Consultas compartidas con tres intentos totales y un evento final. Mutación con 401 que conserva su callback local de error.

Payload JSON serializado con campos legítimos conservados y campos sensibles filtrados. Omisión de payloads de login y registro y del recurso auth. Ruta global de filterEvent con payload Axios. Referencias circulares y cuerpo no JSON. Ausencia de cuerpo. Cliente desactivado sin envío.

Compilación

npm run build pasó después de la última implementación. La salida avisó de Node 20.18.3 frente al mínimo indicado por Vite de 20.19+ o 22.12+, y de un bundle superior a 500 kB. El build terminó, pero los avisos no equivalen a mejoras ya realizadas.

No demostrado por estas pruebas

No hay prueba de extremo a extremo de las pantallas de recuperación ni de navegación real. No se confirmó la recepción del payload final en el panel remoto. El test no verifica toda la configuración de sentry.ts: prueba funciones y un cliente construido específicamente para la prueba.

La sincronización de login/logout, los spans reales del navegador, bloqueos de red, límites de cuota y el comportamiento de CORS deben comprobarse funcionalmente. No se debe presentar la compilación o el transporte simulado como prueba de entrega remota.

13. Comprobación y diagnóstico

Procedimiento de validación manual

1. Confirmar que el entorno de ejecución tiene VITE_SENTRY_DSN del proyecto correcto. Si cambia .env, reiniciar el servidor de desarrollo. No copiar su contenido completo en un informe ni compartir credenciales.
2. Abrir la aplicación, iniciar sesión y realizar una operación. Para comprobar el fallo de productos, usar un escenario controlado que responda con error y revisar la petición en Network.
3. Confirmar estado, método y ruta. Un 401 describe rechazo de autenticación; comprobar localmente si existe Authorization, sin compartir el valor. Sentry no determina por sí solo si el token ha expirado.
4. Revisar si se producen envíos a Sentry y si el navegador los bloquea. Una respuesta correcta al envelope es una pista de transporte; revisar su tipo de contenido o el evento en el proyecto para confirmar que se trata de una excepción.
5. Buscar eventos recientes en el proyecto y entorno adecuados. Revisar una incidencia existente, su última ocurrencia y el evento individual. Comparar operation y resource para identificar la acción.
6. Verificar request_payload.body en una petición JSON fallida no relacionada con autenticación. Confirmar campos filtrados y ausencia de cabeceras. Probar logout y otro error para comprobar que desaparece el usuario asociado.

Si no aparece un error

Comprobar DSN, entorno, filtros de búsqueda, rango horario, bloqueador del navegador y respuestas del transporte. Después confirmar que la llamada usa el QueryClient compartido o reportError, y que no se absorbió la excepción antes de llegar a esa capa.

Una petición que responde 200 no se convierte en fallo solo porque la interfaz muestre un mensaje. Una consulta con reintentos puede tardar en reportarse. Volver a pasar el mismo objeto Error por reportError tampoco genera un segundo evento.

Si aparece el error pero no el payload

Comprobar si realmente había cuerpo, si Axios guardó config.data, si la ruta era de autenticación, si el cuerpo era JSON o si se excedieron límites. Los datos de la URL no se convierten en payload y se eliminan sus query params.

Si faltan trazas, recordar el muestreo de producción y que no toda petición constituye una traza independiente. Ningún paso de esta guía requiere dejar un error de prueba permanente en main.tsx.

14. Inventario y mantenimiento

Archivos de infraestructura

`src/infrastructure/monitoring/sentry.ts`: configuración, instrumentación de navegación, filtros y sincronización del usuario.

`src/infrastructure/monitoring/errors.ts`: captura común, deduplicación, metadatos HTTP y transformación de excepciones.

`src/infrastructure/monitoring/payload.ts`: exclusión de autenticación, extracción del cuerpo JSON y sanitización limitada.

`src/infrastructure/monitoring/privacy.ts`: filtros finales para eventos, breadcrumbs y spans.

`src/infrastructure/monitoring/queryClient.ts`: callbacks comunes de `QueryCache` y `MutationCache`.

Puntos de conexión

`src/main.tsx` inicializa la monitorización y configura las fronteras y callbacks de React. `src/app/routes.tsx` envuelve el router e instala `errorElement`. `src/app/provider/QueryProvider.tsx` comparte el cliente configurado.

`src/presentation/components/ErrorRecovery.tsx` implementa la recuperación visual y captura de rutas y raíz. Los hooks `useProducts`, `useTasks`, `useCategories`, `useLogin` y `useRegister` aportan las etiquetas de recurso y operación. `tests/sentry.test.mjs` verifica las principales reglas.

Reglas al ampliar el proyecto

Usar el cliente común, declarar meta estable y evitar capturas repetidas en cada capa. No adjuntar `AxiosError`, store, cabeceras o variables de formularios completos. Para errores manejados fuera de `TanStack Query`, decidir explícitamente si deben pasar por `reportError`.

Al añadir endpoints de autenticación, revisar exclusiones de payload. Al cambiar la política de errores o privacidad, actualizar pruebas. Al actualizar Sentry, revisar tipos y comportamiento de sus integraciones: los cambios de versión ya afectaron las opciones y el formato de spans durante esta implementación.

Decisiones que permanecen abiertas

No se implementó instrumentación del backend, propagación distribuida, source maps, releases, alertas remotas, Session Replay ni políticas de retención del servicio. Tampoco una lista de campos de payload permitidos por recurso.

Estas capacidades son ampliaciones posibles, no requisitos para que la captura frontend actual funcione. La siguiente comprobación práctica es validar con un evento nuevo en el navegador que el proyecto remoto recibe el error, sus etiquetas y el cuerpo depurado esperado.